

Foundations for programming languages

8: Types

Isao Sasano

Typed languages

- Statically-typed languages
 - Check the consistency of types in compile time
 - (ex.) C, Java, Pascal
- Dynamically-typed languages
 - Check the consistency of types in runtime
 - (ex.) Lisp, Scheme, Python, Ruby, JavaScript

Type checking

In statically typed languages, consistency of programs is checked in compile time with respect to the typing constraint of the language. Type checking partially ensures the correctness of programs and decreases the runtime errors.

Types are *static semantics* (information that is obtained without executing programs) and type checking is a kind of static program analysis.

Type checking is usually performed after the parsing and elaboration (desugaring) phase in compilers.

Small expression language

Terms:

$\langle t \rangle ::= \text{true} \mid \text{false} \mid \text{if } \langle t \rangle \text{ then } \langle t \rangle \text{ else } \langle t \rangle \mid 0$
 $\mid \text{succ } \langle t \rangle \mid \text{pred } \langle t \rangle \mid \text{iszero } \langle t \rangle$

Values:

$\langle v \rangle ::= \text{true} \mid \text{false} \mid \langle nv \rangle$

$\langle nv \rangle ::= 0 \mid \text{succ } \langle nv \rangle$

(Small step) operational semantics

$$\frac{}{\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2} \text{ (E-IFT)}$$

$$\frac{}{\text{if false then } t_2 \text{ else } t_3 \rightarrow t_3} \text{ (E-IFF)}$$

$$\frac{t_1 \rightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3} \text{ (E-IF)}$$

$$\frac{t_1 \rightarrow t_1'}{\text{succ } t_1 \rightarrow \text{succ } t_1'} \text{ (E-S)} \quad \frac{}{\text{pred } 0 \rightarrow 0} \text{ (E-PZ)}$$

$$\frac{}{\text{pred (succ } nv) \rightarrow nv} \text{ (E-PS)} \quad \frac{t_1 \rightarrow t_1'}{\text{pred } t_1 \rightarrow \text{pred } t_1'} \text{ (E-P)}$$

$$\frac{}{\text{iszero } 0 \rightarrow \text{true}} \text{ (E-ISZZ)} \quad \frac{}{\text{iszero (succ } nv) \rightarrow \text{false}} \text{ (E-ISZS)}$$

$$\frac{t_1 \rightarrow t_1'}{\text{iszero } t_1 \rightarrow \text{iszero } t_1'} \text{ (E-ISZ)}$$

Exercise

Find a term t such that

$\text{pred} (\text{succ} (\text{pred } 0)) \rightarrow t$

and construct the proof tree for that.

$$\frac{\dots}{\text{pred} (\text{succ} (\text{pred } 0)) \rightarrow \dots}$$

$t_1 \rightarrow t_2$ is read as “ t_1 evaluates to t_2 in one step”.

Types

Type expressions:

$\langle T \rangle ::= \text{Bool} \mid \text{Nat}$

Intuitively speaking, we'd like to divide the set of terms into two:

- The set of terms that evaluate to true or false in zero or more steps --- Bool type
- The set of terms that evaluate to *nv* in zero or more steps --- Nat type

This is a rough idea. We will modify this later.

A type system for the language

$$\begin{array}{c} \frac{}{\text{true} : \text{Bool}} \text{ (T-T)} \qquad \frac{}{\text{false} : \text{Bool}} \text{ (T-F)} \\[1em] \frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{ (T-IF)} \qquad \frac{}{0 : \text{Nat}} \text{ (T-Z)} \\[1em] \frac{t : \text{Nat}}{\text{succ } t : \text{Nat}} \text{ (T-S)} \qquad \frac{t : \text{Nat}}{\text{pred } t : \text{Nat}} \text{ (T-P)} \\[1em] \frac{t : \text{Nat}}{\text{iszero } t : \text{Bool}} \text{ (T-ISZ)} \end{array}$$

A type system is a set of *typing rules*.

An example

A type judgement

if iszero 0 then 0 else pred 0: Nat
holds.

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{ (T-Z)}}{\text{iszero } 0 : \text{Bool}} \text{ (T-ISZ)} \quad \frac{\frac{}{0 : \text{Nat}} \text{ (T-Z)}}{\text{pred } 0 : \text{Nat}} \text{ (T-P)}}{\text{if iszero } 0 \text{ then } 0 \text{ else pred } 0 : \text{Nat}} \text{ (T-IF)}$$

A derivation tree for the judgement

Exercise

Find a type T such that the following type judgement holds.

if true then pred 0 else succ 0: T

Then derive the judgement by using the typing rules.

...

if true then pred 0 else succ 0: ...

(T-IF)

An example of type judgement that does not hold

A type judgement

if iszero 0 then 0 else false: T

does not hold for any T (either of Bool or Nat).

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{ (T-Z)}}{\text{iszero } 0 : \text{Bool}} \text{ (T-ISZ)} \quad \frac{\frac{}{0 : \text{Nat}} \text{ (T-Z)}}{\text{false} : \text{Bool}} \text{ (T-F)}}{\text{if iszero 0 then 0 else false : ...}} \text{ (T-IF)}$$

The typing rule (T-IF) requires that the two types colored red are the same, but they are different. So (T-IF) can not be applied.

An example of type judgement that does not hold (Cont.)

Actually the term

if iszero 0 then 0 else false

can be evaluated to 0 without causing any error, but we exclude this kind of terms to assure that type checked terms do not cause any error concerning types.

Properties of type systems

- *Soundness* --- type checked programs do not cause any error concerning types at runtime.
- *Completeness* --- programs that do not cause any error concerning types at runtime are type checked.

We cannot get both of these two for *Turing complete* statically typed languages.

Soundness is much more important since it concerns security of the programs (*type safety*). So we abandon the completeness.

Type systems should be *conservative*.

Properties of type systems (Cont.)

The term

`if iszero 0 then 0 else false`

does not go wrong but does not type check.

If we'd like to write this kind of programs,
we can use dynamically typed languages.

In this sense, dynamically typed languages
are more flexible than statically typed
languages.

Soundness

Soundness are divided into two properties.

- *Progress*: If $t : T$ then t is a value or t can be evaluated one step.
- *Preservation*: If $t : T$ and $t \rightarrow t'$ then $t' : T$.

These two can be proved by induction on the derivation trees of type judgements, but we omit the proof.
(Inductive proofs are outside the scope of this class.)

Function types

We can consider *function types*, if the language provides functions.

$$\langle T \rangle ::= \text{Bool} \mid \text{Nat} \mid \langle T \rangle \rightarrow \langle T \rangle$$

“ $\rightarrow$ ” is called a *type constructor*.

Examples of function types:

$\text{Bool} \rightarrow \text{Bool}$, $\text{Nat} \rightarrow \text{Bool}$, $(\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Nat}$,
 $((\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Bool}) \rightarrow (\text{Nat} \rightarrow \text{Nat})$, etc.

A small functional language (lambda expressions)

$\langle t \rangle ::= \langle id \rangle$	variables
$\quad (\lambda \langle id \rangle. \langle t \rangle)$	lambda abstractions
$\quad (\langle t \rangle \langle t \rangle)$	function applications
$\langle id \rangle ::= x \mid y \mid z \mid \dots$	

Examples:

$x, y, (\lambda x. x), (x y), (\lambda x. (\lambda y. (x y))), \dots$

Lambda abstractions

λ abstractions were studied by Alonzo Church in the paper of *lambda calculus* in 1936 to investigate what are computable functions.

Intuitively, $(\lambda x. x)$ represents a function that takes x and returns x . In mathematics, $g(x) = x$ and $g(y) = y$ define the same function on, e.g., real numbers, which is described as $(\lambda x. x)$.

Nowadays λ abstractions are used in functional languages and incorporated to recent languages like Python, Ruby, JavaScript, etc. (a.k.a. *no name* or *anonymous* functions)

Alonzo Church,

“An unsolvable problem of elementary number theory”,

American Journal of Mathematics, vol. 58 , pp. 345-363, 1936.

A type system with function types

One simple way is to attach type expressions to the formal argument of lambda abstractions.

$\langle t \rangle ::= \langle \text{id} \rangle$	variables
$\quad (\lambda \langle \text{id} \rangle : \langle T \rangle . \langle t \rangle)$	lambda abstractions
$\quad (\langle t \rangle \langle t \rangle)$	function applications

This kind of languages are referred to as *explicitly typed* languages, as opposed to *implicitly typed* languages. Real functional languages such as Haskell, Standard ML, OCaml, etc. are hybrid of these two.

A type system with function types (Cont.)

We now add expressions about natural numbers.

$$\begin{aligned} \langle t \rangle ::= & \langle \text{id} \rangle \mid (\lambda \langle \text{id} \rangle : \langle T \rangle . \langle t \rangle) \mid (\langle t \rangle \langle t \rangle) \\ & \mid 0 \mid \text{succ } \langle t \rangle \mid \text{pred } \langle t \rangle \end{aligned}$$

We use the following type expressions:

$$\langle T \rangle ::= \text{Nat} \mid \langle T \rangle \rightarrow \langle T \rangle \quad (\text{Bool type excluded now})$$

A type system with function types (Cont.)

$$\begin{array}{c}
 \frac{x : T \in \Gamma}{\Gamma \vdash x : T} \text{ (T-VAR)} \qquad \frac{\Gamma \{x : T_1\} \vdash t_2 : T_2}{\Gamma \vdash (\lambda x : T_1. t_2) : T_1 \rightarrow T_2} \text{ (T-ABS)} \\
 \\
 \frac{\Gamma \vdash t_1 : T_2 \rightarrow T_1 \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash (t_1 t_2) : T_1} \text{ (T-APP)} \\
 \\
 \frac{}{\Gamma \vdash 0 : \text{Nat}} \text{ (T-Z)} \quad \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{succ } t : \text{Nat}} \text{ (T-S)} \quad \frac{\Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{pred } t : \text{Nat}} \text{ (T-P)}
 \end{array}$$

We introduce type environments:

$$\langle \text{tenv} \rangle ::= \emptyset \mid \langle \text{tenv} \rangle \{ \langle \text{id} \rangle : \langle T \rangle \}$$

An example

A type judgement

$\emptyset \vdash ((\lambda x: \text{Nat}. x) 0) : \text{Nat}$

holds.

$$\frac{\frac{\frac{x: \text{Nat} \in \emptyset\{x: \text{Nat}\}}{\emptyset\{x: \text{Nat}\} \vdash x : \text{Nat}} \text{ (T-VAR)}}{\emptyset \vdash (\lambda x: \text{Nat}. x) : \text{Nat} \rightarrow \text{Nat}} \text{ (T-ABS)} \quad \frac{}{\emptyset \vdash 0 : \text{Nat}} \text{ (T-Z)}}{\emptyset \vdash ((\lambda x: \text{Nat}. x) 0) : \text{Nat}} \text{ (T-APP)}$$

A derivation tree for the judgement

Exercise

Find a type T such that the following type judgement holds.

$$\emptyset \vdash ((\lambda x: \text{Nat. pred } x) (\text{succ } 0)) : T$$

Then derive the judgement by using the typing rules.

...

$$\emptyset \vdash ((\lambda x: \text{Nat. pred } x) (\text{succ } 0)) : \dots$$

Properties

Some properties such as soundness hold for this type system similarly to the previous type system. We omit it.